

ABSTRACT

Coal mining is one of the most hazardous occupations in the industrial world.

Miners are continuously exposed to potential dangers such as toxic gases (methane, carbon monoxide), poor lighting, high humidity, and the risk of accidents due to falls or collapses.

Traditional helmets offer only physical protection and fail to provide real-time feedback on environmental and physical conditions.

To overcome these limitations, this project proposes the design and implementation of a Smart Helmet for Coal Mine Workers using IoT (Internet of Things) technology.

The system continuously monitors environmental parameters including temperature, humidity, air quality, and light intensity, while also detecting falls and tracking worker location using GPS.

The data collected from sensors — DHT11, MQ135, LDR, MPU6050, and GPS module — is processed by an ESP32 microcontroller, which has built-in Wi-Fi capability.

The ESP32 transmits this data in real time to a web-based dashboard, allowing supervisors to monitor the condition of workers remotely.

If hazardous conditions are detected, the system immediately triggers buzzer and LED alerts, providing both local and remote warnings.

This Smart Helmet demonstrates how IoT can be integrated with wearable technology to improve worker safety, enhance real-time awareness, and reduce fatalities in mining operations.

1.INTRODUCTION

1.1 Background

Coal mining plays a crucial role in powering modern industries, but it remains one of the most dangerous professions.

According to reports by international safety organizations, thousands of miners lose their lives each year due to accidents and toxic gas exposure.

Traditional safety measures such as ventilation systems and handheld detectors are often inadequate and unreliable.

With the advent of IoT, it is now possible to monitor environmental conditions in real time and transmit information wirelessly.

IoT allows the connection of sensors, controllers, and cloud platforms to form intelligent systems capable of monitoring and decision-making.

1.2 Problem Statement

Conventional helmets provide protection against physical injuries but cannot detect environmental hazards like poor air quality or low visibility.

In the absence of real-time monitoring, accidents go unnoticed until it's too late.

Moreover, in deep underground mines, communication with workers is often limited.

There is also no mechanism to track the location of workers trapped inside after a collapse.

Hence, there is a pressing need for a Smart Helmet that can monitor, alert, and transmit data wirelessly to ensure worker safety.

1.3 Objectives

The objectives of this project are:

1. To design a smart helmet capable of detecting temperature, humidity, air quality, and light intensity.
2. To integrate fall detection using an accelerometer (MPU6050).
3. To incorporate GPS for tracking the real-time location of workers.

- 4 To transmit live data to a web dashboard using Wi-Fi (ESP32).
- . To provide local alerts through LED and buzzer when unsafe conditions occur.
- 5 To create a low-cost, portable, and scalable safety solution for industrial use.

1.4 Scope

The system focuses on coal mine worker safety, but its application can extend to other hazardous industries such as:

- ☐ Underground construction sites
- ☐ Oil refineries
- ☐ Firefighting and disaster management
- ☐ Chemical and gas industries
- ☐



Figure 1: Typical Coal Mine Environment

This project lays the groundwork for intelligent wearable devices in industrial safety monitoring.

2.LITERATURE REVIEW

2.1 Previous Work

A number of systems have been proposed for miner safety in the past, but most were limited by wired communication, poor scalability, and lack of integration.

Wired systems often fail in rough mining environments due to cable damage.

Recent developments have utilized wireless sensor networks (WSNs) and IoT, which have proven more effective in monitoring parameters such as temperature, humidity, and gas levels. However, most systems lack features like GPS tracking or fall detection, which are crucial for ensuring complete safety.

2.2 Comparative Analysis

System	Parameters Monitored	Communication Type	Limitations
Wired Gas Detection	Gas only	Wired	Limited range
Bluetooth Helmet	Temperature, Light	Bluetooth	Short range, no GPS
GSM Helmet	Gas, Fall Detection	GSM	Expensive, higher power Reliable, scalable
Proposed IoT Helmet	Gas, Temp, Light, GPS	Wi-Fi	

Table 1: Comparison of existing systems and proposed design

2.3 Research Gap

Existing systems focus on environmental data but ignore real-time alerts and remote accessibility.

Moreover, integrating multiple sensors into a wearable device remains a challenge.

The proposed Smart Helmet combines multiple parameters, real-time transmission, and live visualization on a web interface.



Figure 2: Previous design concept of a smart safety helmet.

3 . SYSTEM DESIGN AND METHODOLOGY

3.1 Overview

The proposed system consists of the following main components:

- ❑ Sensor Unit (DHT11, MQ135, LDR, MPU6050, GPS)
- ❑ ESP32 Microcontroller
- ❑ Buzzer and LED
- ❑ Web Dashboard

The ESP32 collects data, checks thresholds, and transmits information to a cloud server. Alerts are triggered both locally (on the helmet) and remotely (on the web dashboard).

3.2 Block Diagram

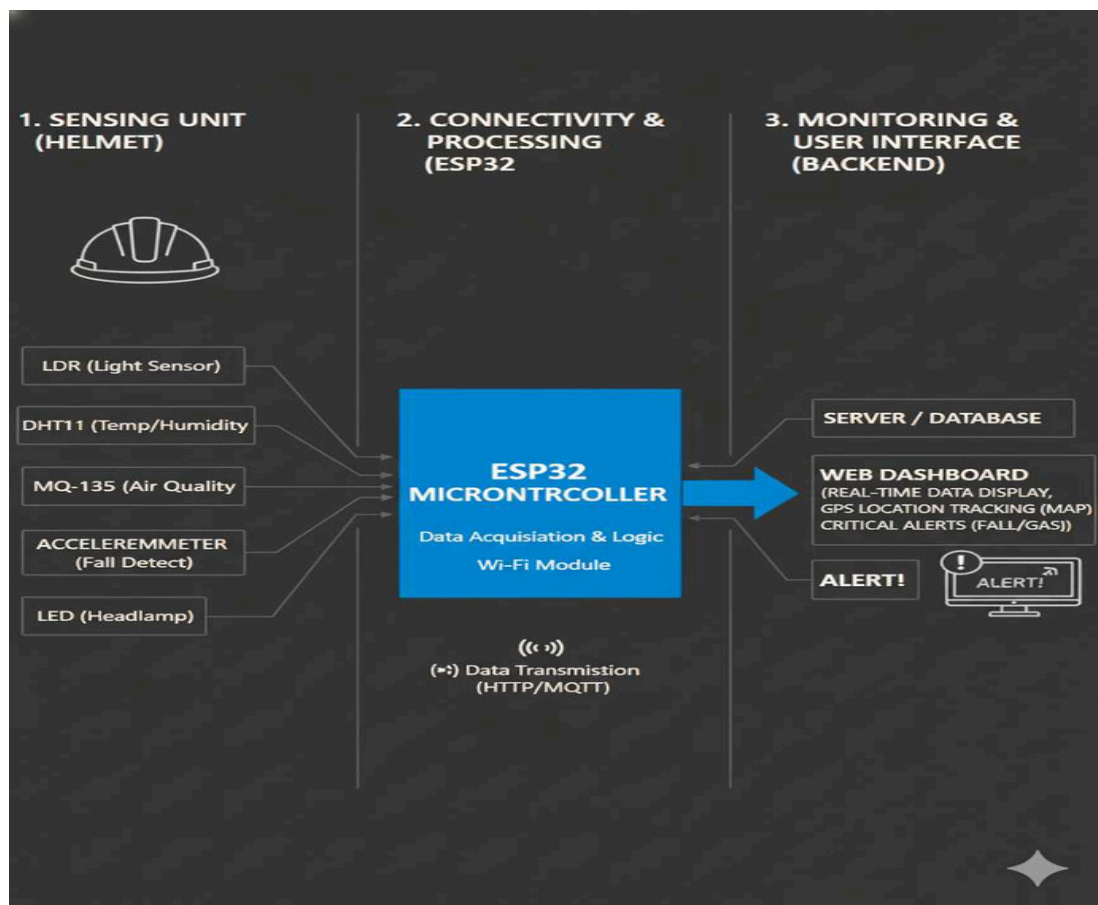


Figure 3: Block Diagram of Smart Helmet System

3.3 Working Principle

The system operates in three major stages:

1. Data Collection: Sensors measure temperature, humidity, light intensity, and air quality.

2. Processing: ESP32 processes sensor readings and compares them with preset safety thresholds.
3. Transmission and Alert: If unsafe conditions are detected, buzzer and LED alerts are activated, and data is sent to the web dashboard for monitoring.

3.4 Circuit Diagram

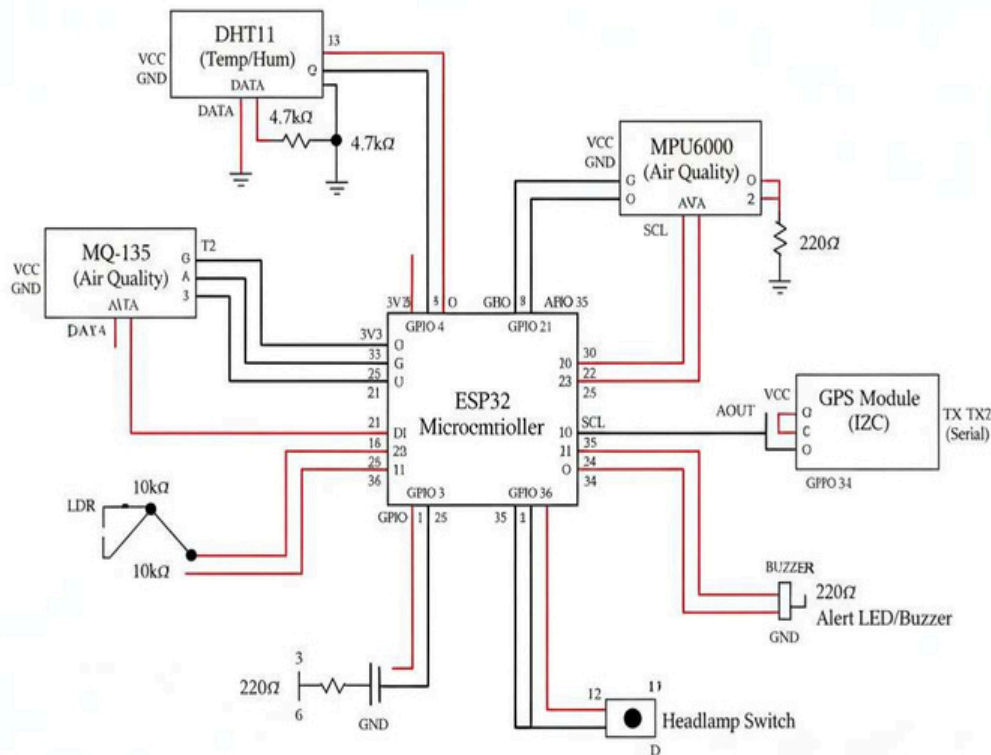
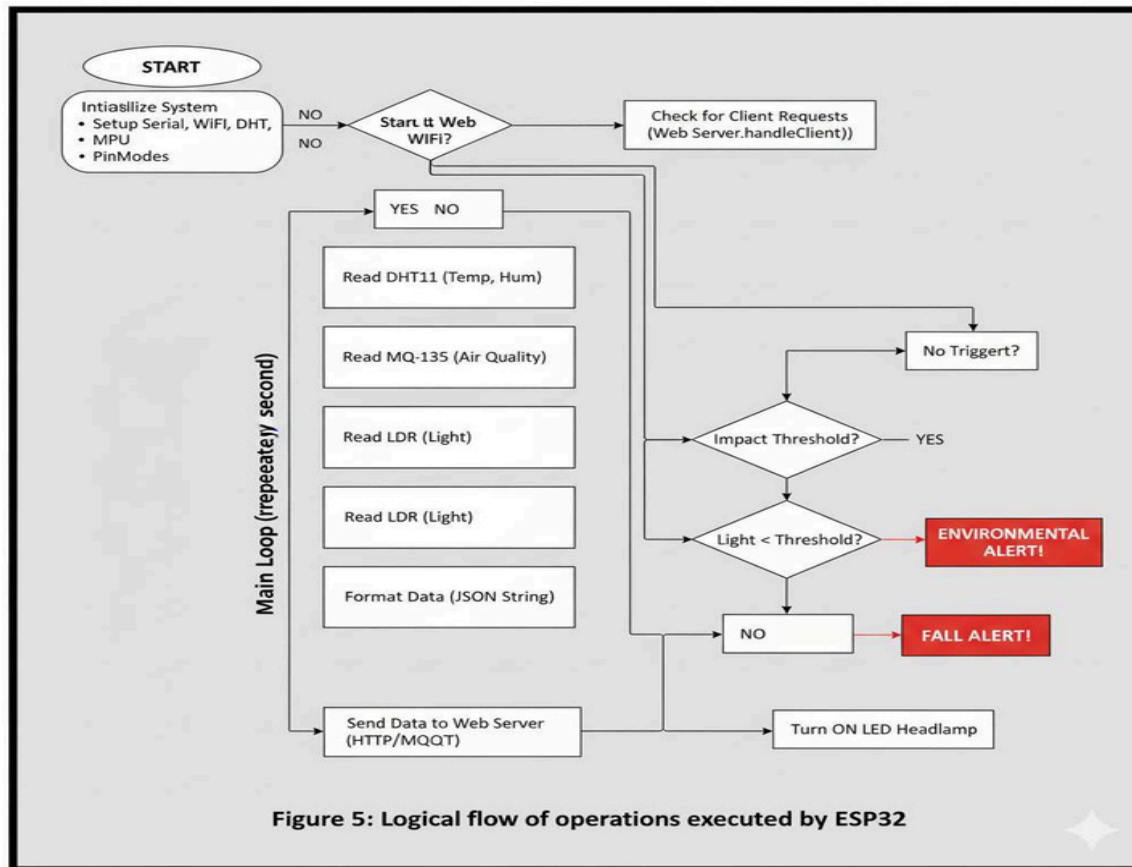


Figure 4: Circuit connections between sensors and ESP32 microcontroller.

3.5 Flowchart



4.HARDWARE AND SOFTWARE IMPLEMENTATION

4.1 Hardware Description

Component	Specification	Description
ESP32	Wi-Fi, Dual-core MCU	Controls sensors and sends data
DHT11	Temp: 0–50°C, Humidity: 20–90%	Measures temperature & humidity
MQ135	Range: 10–3000 ppm	Detects harmfull gases
LDR	Analog	Measures light level

MPU6050	3-axis accelerometer	Detects fall and vibration
GPS Module	NEO-6M	Tracks real-time location
LED/Buzzer	Digital Output	Alerts user in hazard
Li-ion Battery	3.7V, 1200mAh	Powers the helmet

Table 2: Specifications of components used in Smart Helmet.

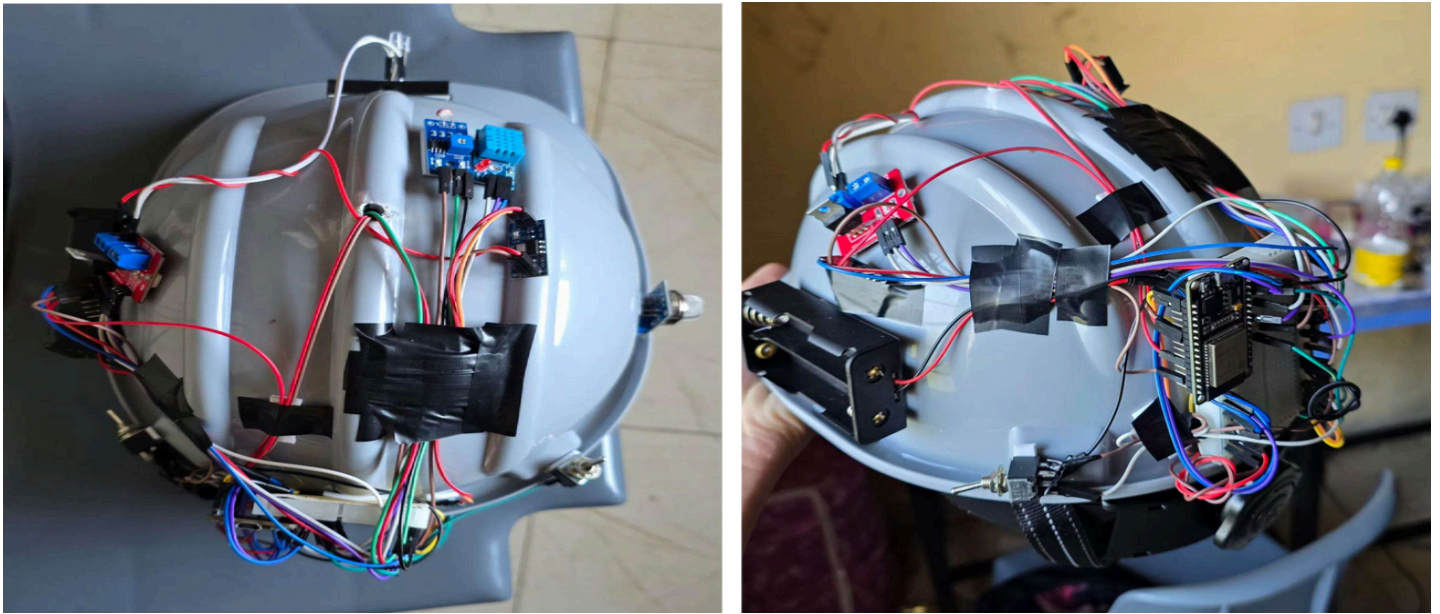


Figure 6: Assembled Prototype of Smart Helmet

4.2 Software Description

The software development involves programming the ESP32 in Arduino IDE.

The microcontroller is coded to read sensor data, process it, and send it to a web server using HTTP or MQTT protocols.

The web dashboard is developed using HTML, CSS, and JavaScript, and updates in real-time. Cloud platforms such as ThingSpeak or Firebase can also be integrated for data storage.

4.3 Algorithm

1. Start and initialize all sensors.
2. Connect ESP32 to Wi-Fi.
3. Continuously read values from sensors.
4. If threshold crossed → trigger alert.
5. Transmit data to server every 5 seconds.
6. Display readings on the web dashboard.

5.RESULTS AND DISCUSSION

5.1 Testing Environment

The prototype was tested under simulated mining conditions with varying light intensity and gas concentration.

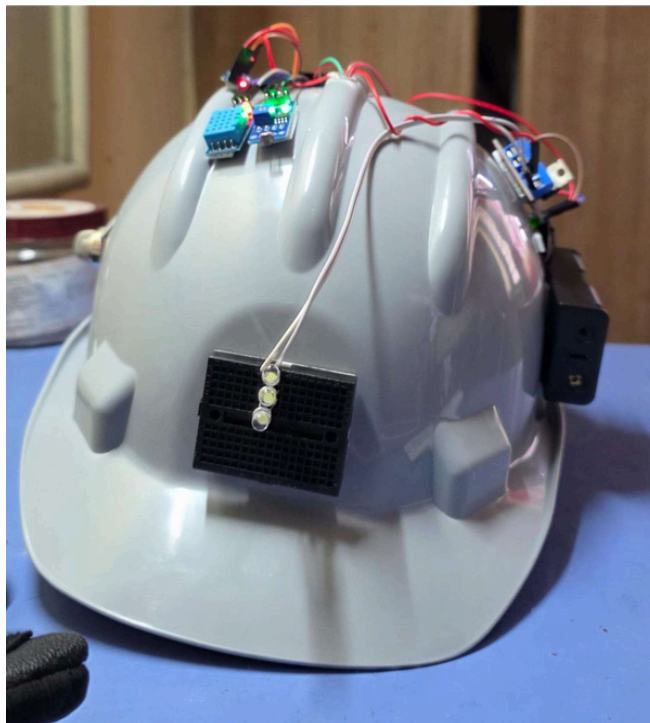


Fig 7.a Smart Helmet in light

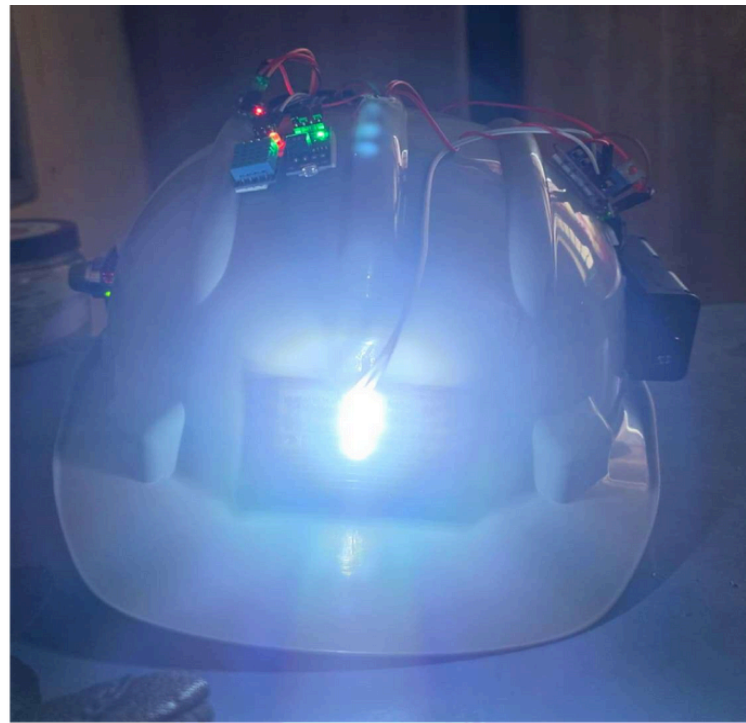


Fig 7.b Smart Helmet in Dark

5.2 Sample Readings



5.3 Analysis

The system responded accurately to variations in gas concentration, light, and falls.

The dashboard displayed updates within 2–3 seconds of data transmission.

The buzzer and LED provided effective local warnings.

6.APPLICATIONS AND LIMITATIONS

6.1 Applications

- ❑ Coal mine and tunnel worker safety monitoring.
- ❑ Oil refineries and chemical industries.
- ❑ Construction sites and high-risk zones.
- ❑ Firefighter safety and rescue operations.

6.2 Limitations

- ❑ Wi-Fi signal loss in deep underground areas.
- ❑ Limited battery backup.
- ❑ Calibration required for accurate gas detection.
- ❑ No GSM connectivity in the current version.

7.CONCLUSION AND FUTURE SCOPE

The Smart Helmet successfully demonstrates how IoT can enhance industrial safety through wearable technology.

It provides real-time monitoring, quick alerts, and remote visibility — significantly reducing risks in mining environments.

Future Scope:

1. Integration with GSM or LoRa for underground communication.
2. Solar-powered charging system.
3. Integration with AI for predictive hazard analysis.
4. Mobile application for real-time alerts.

8. REFERENCES

- R. S. Shinde, M. B. Patil, and A. P. Rane,
“IoT Based Smart Helmet for Coal Mine Safety,”
IEEE International Conference on Intelligent Computing and Control Systems (ICICCS),
Madurai, India, pp. 987–992, Jun. 2021.
- P. K. Sahu and S. K. Singh,
“Real-Time Environmental Monitoring for Underground Mines Using IoT,”
IEEE Access, vol. 9, pp. 113245–113255, 2021, doi: 10.1109/ACCESS.2021.3098765.
- A. Kumar, R. Gupta, and S. Verma,
“Design and Implementation of an IoT-Enabled Safety Helmet for Industrial Workers,”
IEEE Sensors Journal, vol. 22, no. 14, pp. 14376–14384, Jul. 2022, doi:
10.1109/JSEN.2022.3176554.

9.APPENDIX

ESP 32 Code +HTML Code:

```
#include <Wire.h>

#include <TinyGPS++.h>

#include <HardwareSerial.h>

#include <Adafruit_MPU6050.h>

#include <Adafruit_Sensor.h>

#include "DHT.h"


//===== PIN DEFINITIONS =====

//Sensor Pins

#define MQ135_PIN 34      // MQ-135 Gas Sensor (Analog)

#define DHT_PIN 25        // DHT11/HW-036 Temperature & Humidity

#define LDR_PIN 35        // LDR Light Sensor (Analog)

#define MPU6050_SDA 21     // I2C SDA for MPU6050

#define MPU6050_SCL 22     // I2C SCL for MPU6050


//GPS Pins

#define GPS_RX 16          // GPS TX -> ESP32 RX

#define GPS_TX 17          // GPS RX -> ESP32 TX


//Control Pins

#define LED_STRIP_PIN 27   // MOSFET SIG pin for 12V LED Strip

#define BUZZER_PIN 26      // Buzzer for alerts

#define EMERGENCY_BTN_PIN 14 // Emergency push button

#define TOGGLE_SW1_PIN 13   // Toggle switch 1 (System Enable)

#define TOGGLE_SW2_PIN 12   // Toggle switch 2 (Spare/Mode)

#define STATUS_LED_PIN 2    // Built-in status LED
```

```
// ===== SENSOR THRESHOLDS =====

#define GAS_THRESHOLD 400    // MQ-135 air quality threshold

#define TEMP_THRESHOLD 35.0 // Temperature threshold (°C)

#define HUMIDITY_LOW 30.0    // Low humidity threshold

#define HUMIDITY_HIGH 80.0 // High humidity threshold

#define LDR_DARK_THRESHOLD 1000 // LDR value ABOVE which it's dark (CORRECTED)

#define FALL_THRESHOLD 2.5    // Acceleration threshold for fall (g)
```

```
//===== SENSOR OBJECTS =====
```

```
DHT dht(DHT_PIN, DHT11);

TinyGPSPlus gps;

HardwareSerial gpsSerial(1);

Adafruit_MPU6050 mpu;
```

```
//===== GLOBAL VARIABLES =====
```

```
//Sensor Data
```

```
float temperature = 0.0;

float humidity = 0.0;

int gasLevel = 0;

int ldrValue = 0;

bool isDark = false;

float accelX, accelY, accelZ;

float gyroX, gyroY, gyroZ;
```

```
//GPS Data
```

```
double latitude = 0.0;

double longitude = 0.0;

String gpsDate = "";

String gpsTime = "";

intsatellites = 0;
```

```
//System Status
```

```

bool emergencyAlert = false;

bool fallDetected = false;

bool gasAlert = false;

bool tempAlert = false;

bool ledStripOn = false;

bool systemEnabled = true;


//Timing variables

unsigned long lastSensorRead = 0;

unsigned long lastGPSRead = 0;

unsigned long lastDisplayUpdate = 0;

const unsigned long sensorInterval = 2000; //2seconds

const unsigned long gpsInterval = 5000; //5seconds

const unsigned long displayInterval = 3000; //3seconds


//===== SETUP FUNCTION =====

void setup() {

  Serial.begin(115200);

  Serial.println("\n=====");

  Serial.println("ESP32 SMART HELMET INITIALIZATION");

  Serial.println("Version 1.1 - LDR Logic Corrected");

  Serial.println("=====");


  //Initialize I2C for MPU6050

  Wire.begin(MPU6050_SDA, MPU6050_SCL);


  //Initialize Pin Modes

  pinMode(LED_STRIP_PIN, OUTPUT);

  pinMode(BUZZER_PIN, OUTPUT);

  pinMode(STATUS_LED_PIN, OUTPUT);

  pinMode(EMERGENCY_BTN_PIN, INPUT_PULLUP);

  pinMode(TOGGLE_SW1_PIN, INPUT);

```

```
pinMode(TOGGLE_SW2_PIN, INPUT);

pinMode(MQ135_PIN, INPUT);

pinMode(LDR_PIN, INPUT);


//Initial pin states

digitalWrite(LED_STRIP_PIN, LOW);

digitalWrite(BUZZER_PIN, LOW);

digitalWrite(STATUS_LED_PIN, HIGH); // Power indicator ON


//Initialize DHT Sensor

Serial.print("Initializing DHT11 Sensor...");

dht.begin();

Serial.println(" Done!");


//Initialize MPU6050

Serial.print("Initializing MPU6050...");

if(!mpu.begin()) {

    Serial.println(" FAILED!");

    Serial.println("Check MPU6050 wiring!");

}else {

    Serial.println(" Done!");

    mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

    mpu.setGyroRange(MPU6050_RANGE_500_DEG);

    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

}


//Initialize GPS

Serial.print("Initializing GPS Module...");

gpsSerial.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);

Serial.println(" Done!");


//Startup sound - 3 short beeps
```

```

beep(3, 100);

Serial.println("=====");
Serial.println("SMART HELMET SYSTEM READY!");
Serial.println("=====\\n");

delay(1000);
}

//===== MAIN LOOP =====

void loop() {

    unsigned long currentMillis = millis();

    //Read toggle switches
    systemEnabled = digitalRead(TOGGLE_SW1_PIN);

    //Check if system is disabled by toggle switch
    if(systemEnabled == LOW) {

        Serial.println("System DISABLED by Toggle Switch");
        digitalWrite(LED_STRIP_PIN, LOW); // Turn off LED
        digitalWrite(STATUS_LED_PIN, LOW); // Turn off status LED
        delay(1000);
        return;
    }

    //System is enabled - turn on status LED
    digitalWrite(STATUS_LED_PIN, HIGH);

    //Read sensors periodically
    if(currentMillis - lastSensorRead >= sensorInterval) {

        lastSensorRead = currentMillis;
        readAllSensors();
    }
}

```

```
    checkAlertConditions();  
}
```

```
//Read GPS periodically  
  
if(currentMillis - lastGPSRead >= gpsInterval) {  
  
    lastGPSRead = currentMillis;  
  
    readGPSData();  
}
```

```
//Display data periodically  
  
if(currentMillis - lastDisplayUpdate >= displayInterval) {  
  
    lastDisplayUpdate = currentMillis;  
  
    displaySensorData();  
}
```

```
//Check emergency button  
  
if(digitalRead(EMERGENCY_BTN_PIN) == LOW) {  
  
    delay(50); // Debounce  
  
    if(digitalRead(EMERGENCY_BTN_PIN) == LOW) {  
  
        triggerEmergency();  
  
        while(digitalRead(EMERGENCY_BTN_PIN) == LOW) {  
  
            delay(10); // Wait for button release  
  
        }  
  
    }  
  
}
```

```
//Control LED strip based on light level  
  
controlLEDStrip();  
}
```

```
//===== SENSOR READING FUNCTIONS =====  
  
void readAllSensors() {
```

```
//Read DHT11

humidity = dht.readHumidity();

temperature = dht.readTemperature();


if(isnan(humidity) || isnan(temperature)) {

    Serial.println("DHT Read Error!");

    humidity = 0;

    temperature = 0;

}


//Read MQ-135 Gas Sensor

gasLevel = analogRead(MQ135_PIN);


//Read LDR - CORRECTED LOGIC

ldrValue = analogRead(LDR_PIN);

//HIGH LDR value = DARK (LDR has high resistance in dark)

//LOW LDR value = BRIGHT (LDR has low resistance in light)

isDark = (ldrValue > LDR_DARK_THRESHOLD); // REVERSED: > instead of <


//Read MPU6050

sensors_event_t a, g, temp;

mpu.getEvent(&a, &g, &temp);


accelX = a.acceleration.x;

accelY = a.acceleration.y;

accelZ = a.acceleration.z;

gyroX = g.gyro.x;

gyroY = g.gyro.y;

gyroZ = g.gyro.z;


//Check for fall detection

float totalAccel = sqrt(accelX*accelX + accelY*accelY + accelZ*accelZ);
```

```

if(totalAccel > FALL_THRESHOLD * 9.81) { // Convert g to m/s2

    fallDetected = true;

}

}

void readGPSData() {

    while(gpsSerial.available() > 0) {

        gps.encode(gpsSerial.read());

    }

    if(gps.location.isValid()) {

        latitude = gps.location.lat();

        longitude = gps.location.lng();

        satellites = gps.satellites.value();

    }

    if(gps.date.isValid()) {

        gpsDate = String(gps.date.day()) + "/" +

            String(gps.date.month()) + "/" +

            String(gps.date.year());

    }

    if(gps.time.isValid()) {

        gpsTime = String(gps.time.hour()) + ":" +

            String(gps.time.minute()) + ":" +

            String(gps.time.second());

    }

}

//===== ALERT CHECKING =====

void checkAlertConditions() {

    bool alertTriggered = false;

```

```
String alertMessage = "";
```

```
//Check gas levels
```

```
if(gasLevel > GAS_THRESHOLD) {  
  
    gasAlert = true;  
  
    alertTriggered = true;  
  
    alertMessage += "HIGH GAS LEVEL! ";  
  
}else {  
  
    gasAlert = false;  
  
}
```

```
//Check temperature
```

```
if(temperature > TEMP_THRESHOLD) {  
  
    tempAlert = true;  
  
    alertTriggered = true;  
  
    alertMessage += "HIGH TEMPERATURE! ";  
  
}else {  
  
    tempAlert = false;  
  
}
```

```
//Check humidity
```

```
if(humidity < HUMIDITY_LOW || humidity > HUMIDITY_HIGH) {  
  
    alertTriggered = true;  
  
    alertMessage += "ABNORMAL HUMIDITY! ";  
  
}
```

```
//Check fall detection
```

```
if(fallDetected) {  
  
    alertTriggered = true;  
  
    alertMessage += "FALL DETECTED! ";  
  
}
```

```

//Trigger alerts if needed

if(alertTriggered) {

    Serial.println("\n*** ALERT: " + alertMessage + "**");

    soundAlert(5);


    //Flash status LED

    for(int i = 0; i < 3; i++) {

        digitalWrite(STATUS_LED_PIN, LOW);

        delay(100);

        digitalWrite(STATUS_LED_PIN, HIGH);

        delay(100);

    }


    fallDetected = false; // Reset fall detection

}

}


//===== LED STRIP CONTROL (CORRECTED) =====

void controlLEDStrip() {

    //CORRECTED: Automatic LED control based on LDR

    //Higher LDR value = Darker environment

    //Lower LDR value = Brighter environment


    if(isDark && !ledStripOn) {

        digitalWrite(LED_STRIP_PIN, HIGH); // Turn ON 12V LED strip via MOSFET

        ledStripOn = true;

        Serial.println("LED Strip: ON (Dark detected - LDR value: " + String(ldrValue) + ")");

    } else if (!isDark && ledStripOn) {

        digitalWrite(LED_STRIP_PIN, LOW); // Turn OFF LED strip

        ledStripOn = false;

        Serial.println("LED Strip: OFF (Light detected - LDR value: " + String(ldrValue) + ")");

    }
}

```

```
}
```

```
//===== EMERGENCY FUNCTIONS =====
```

```
void triggerEmergency() {
```

```
    emergencyAlert = true;
```

```
    Serial.println("\n!!! EMERGENCY BUTTON PRESSED !!!");
```

```
    Serial.println("Location: Lat=" + String(latitude, 6) +
```

```
        "Lng=" + String(longitude, 6));
```

```
    //FlashLED and sound buzzer repeatedly
```

```
    for(int i=0; i < 10; i++) {
```

```
        digitalWrite(STATUS_LED_PIN, HIGH);
```

```
        digitalWrite(BUZZER_PIN, HIGH);
```

```
        delay(200);
```

```
        digitalWrite(STATUS_LED_PIN, LOW);
```

```
        digitalWrite(BUZZER_PIN, LOW);
```

```
        delay(200);
```

```
    }
```

```
    emergencyAlert = false;
```

```
}
```

```
void soundAlert(int times) {
```

```
    for(int i=0; i < times; i++) {
```

```
        digitalWrite(BUZZER_PIN, HIGH);
```

```
        delay(100);
```

```
        digitalWrite(BUZZER_PIN, LOW);
```

```
        delay(100);
```

```
    }
```

```
}
```

```
void beep(int times, int duration) {
```

```

for (int i = 0; i < times; i++) {

    digitalWrite(BUZZER_PIN, HIGH);

    delay(duration);

    digitalWrite(BUZZER_PIN, LOW);

    delay(duration);

}

}

//===== DISPLAY FUNCTIONS =====

void displaySensorData() {

    Serial.println("\n===== SENSOR READINGS =====");

    Serial.println("Temperature: " + String(temperature, 1) + " °C");

    Serial.println("Humidity: " + String(humidity, 1) + " %");

    Serial.println("Gas Level: " + String(gasLevel) + " ppm");

    Serial.println("Light Level: " + String(ldrValue) + (isDark ? " (DARK)" : " (BRIGHT)"));

    Serial.println("Acceleration: X=" + String(accelX, 2) + " Y=" + String(accelY, 2) + " Z=" + String(accelZ, 2) + " m/s²");

    Serial.println("Gyro: X=" + String(gyroX, 2) + " Y=" + String(gyroY, 2) + " Z=" + String(gyroZ, 2) + " rad/s");

    if(gps.location.isValid()) {

        Serial.println("GPS: Lat=" + String(latitude, 6) + " Lng=" + String(longitude, 6));

        Serial.println("Satellites: " + String(satellites));

        Serial.println("Date: " + gpsDate + " Time: " + gpsTime);

    } else {

        Serial.println("GPS: Waiting for fix...");

    }

    Serial.println("LED Strip: " + String(ledStripOn ? "ON" : "OFF"));

    Serial.println("System: " + String(systemEnabled ? "ENABLED" : "DISABLED"));

    Serial.println("=====\n");

}

```